SmolTulu: Higher Learning Rate to Batch Size Ratios Can Lead to Better Reasoning in SLMs

Sultan Alrashed
Saudi Data & Artificial Intelligence Authority
srashed@sdaia.gov.sa

arXiv:2412.08347v1 [cs.CL] 11 Dec 2024

Abstract

We present SmolTulu-1.7b-Instruct[1], referenced in this report as SmolTulu-DPO-1130, an instruction-tuned language model that adapts AllenAI's Tulu 3 post-training pipeline (Lambert et al., 2024a) to enhance Huggingface's SmolLM2-1.7B base model. Through comprehensive empirical analysis using a 135M parameter model, we demonstrate that the relationship between learning rate and batch size significantly impacts model performance in a task-dependent manner. Our findings reveal a clear split: reasoning tasks like ARC and GSM8K benefit from higher learning rate to batch size ratios, while pattern recognition tasks such as HellaSwag and IFEval show optimal performance with lower ratios. These insights informed the development of SmolTulu, which achieves state-of-the-art performance among sub-2B parameter models on instruction following, scoring 67.7% on IFEval ($\Delta 11\%$), and mathematical reasoning with 51.6% on GSM8K ($\Delta 3.4\%$), with an alternate version achieving scoring 57.1% on ARC ($\Delta 5.4\%$). We release our model, training recipes, and ablation studies to facilitate further research in efficient model alignment, demonstrating that careful adaptation of optimization dynamics can help bridge the capability gap between small and large language models.

1 Introduction

Recent advances in language model post-training have demonstrated remarkable improvements in model capabilities through careful application of supervised finetuning (SFT), preference optimization, and reinforcement learning (Ouyang et al., 2022; Touvron et al., 2023). However, these techniques have primarily been developed and validated on large language models with tens or hundreds of billions of parameters with smaller models being underexplored. The recently released Tulu 3 pipeline provides a comprehensive, open-source approach to post-training, but its effectiveness on significantly smaller models remains unexplored.

Understanding how post-training dynamics scale to smaller models is crucial for democratizing access to high-quality language models and enabling deployment in resource-constrained environments. While (Shallue et al., 2019) demonstrated that the relationship between batch size and training steps follows characteristic patterns across model families, they also found that the maximum useful batch size varies between workloads and depends heavily on model properties. This suggests that smaller models may require fundamentally different optimization strategies than their larger counterparts, especially across different downstream tasks.

Through comprehensive ablations using Huggingface's SmolLM2-135M (Allal et al., 2024) as a base, we demonstrate that the relationship between learning rate and batch size significantly impacts model performance in a task-dependent manner during supervised finetuning. Our findings reveal a clear difference between reasoning and pattern recognition tasks: benchmarks requiring complex reasoning like ARC and GSM8K show optimal performance with higher learning rate to batch size ratios, while pattern recognition tasks such as HellaSwag and IFEval benefit from lower ratios. This aligns with theoretical work by (Masters and Luschi, 2018), who argue that smaller batch sizes, despite their higher gradient variance, enable more frequent parameter updates that can better track the underlying objective's local structure.

Furthermore, (Keskar et al., 2017) demonstrate that optimization trajectories can lead to qualitatively different solutions with varying generalization properties. Our empirical results suggest that higher learning rate to batch size ratios may help smaller models find flatter minima that generalize better to complex reasoning tasks, while lower ratios appear optimal for tasks dominated by pattern

[1]Huggingface link:https://huggingface.co/SultanR/SmolTulu-1.7b-Instruct

recognition. This provides new insight into how careful tuning of optimization parameters might help compensate for limited model capacity in a task-dependent manner.

In this work, we investigate the adaptation of the Tulu 3 post-training pipeline (Lambert et al., 2024a) to enhance SmolLM2-1.7B (Allal et al., 2024), a compact language model with just 1.7 billion parameters. Through a set of ablations, we demonstrate that the relationship between learning rate and batch size plays a crucial role in determining model performance across different types of tasks.

Our key contributions include:

- Empirical evidence demonstrating how learning rate to batch size ratios differentially affect reasoning and pattern recognition capabilities in small language models
- Release of SmolTulu, achieving state-of-the-art performance among sub-2B parameter models on instruction following and mathematical reasoning with verifiably no contamination.
- Theoretical analysis connecting optimization dynamics to task-specific performance in small language models

Our results indicate that while the core techniques from Tulu 3 translate effectively to smaller scales, achieving optimal performance requires careful consideration of how optimization dynamics change with model size and task type. Through these insights, we hope to contribute to the development of more efficient and accessible language models and establish new best practices for training smaller models that diverge from conventional wisdom derived from large-scale training with the learning rate linearly adjusted to the batch size.

2 Related Works

2.1 Post-Training Recipes and Instruction Tuning

The development of modern post-training pipelines began with InstructGPT (Ouyang et al., 2022), which established the core supervised finetuning (SFT) and reinforcement learning from human feedback (RLHF) workflow. This approach has been widely adopted and adapted in open-source efforts, notably through works like Alpaca (Taori et al., 2023) and Vicuna (Chiang et al., 2023), which demonstrated the viability of instruction tuning with synthetic and user-generated data respectively. More recent work has focused on improving these pipelines, with projects like Tulu (Ivison et al., 2023) and Zephyr-β (Tunstall et al., 2023), culminating in comprehensive open pipelines like Tulu 3 (Lambert et al., 2024a).

2.2 Direct Preference Optimization

Direct Preference Optimization (DPO) (Rafailov et al., 2024) introduced a simplified approach to preference learning that eliminates the need for a separate reward model and complex RL training loops. This has been followed by various refinements and alternatives, including SLiC-HF (Zhao et al., 2023) and SimPO (Meng et al., 2024), which further simplified the training process while maintaining or improving performance. These developments have made preference learning more accessible and computationally efficient, particularly important for resource-constrained settings.

2.3 Verifiable Rewards and Reinforcement Learning

Recent work has explored using verifiable outcomes to improve model capabilities, particularly in domains like mathematics and coding. STaR (Zelikman et al., 2022) pioneered the use of self-taught reasoning to improve mathematical capabilities, while TRICE (Phan et al., 2023) developed alternative approaches to learning from verifiable solutions. VinePPO (Kazemnejad et al., 2024) specifically targeted mathematical reasoning through reinforcement learning, demonstrating the effectiveness of RL approaches when ground truth answers are available. This line of work has shown particular promise in improving specific capabilities while maintaining model generality. Reinforcement Learning with Verifiable Rewards (RLVR) extends traditional RL approaches by using deterministic binary rewards based on ground truth outcomes rather than learned reward models(Lambert et al., 2024a). This approach has shown particular promise in domains with verifiable solutions like mathematics and programming, where correct outputs can be automatically validated.

2.4 Learning Rate and Batch Size Studies

The relationship between learning rate and batch size has been a subject of extensive study in deep learning. Foundational work by (Goyal et al., 2018) established key principles for large-batch training,

demonstrating that learning rate should be scaled proportionally with batch size to maintain training stability. (Smith et al., 2018) later proposed the counterintuitive strategy of increasing batch size instead of decaying learning rate, showing that this approach could achieve similar convergence while simplifying hyperparameter tuning.

More recent work has focused on understanding these relationships in the context of transformer architectures. (Kaplan et al., 2020) provided empirical evidence that optimal training dynamics vary with model scale, while (McCandlish et al., 2018) developed theoretical frameworks for understanding how batch size affects training efficiency. These insights have been particularly relevant for language models, where the interaction between learning rate, batch size, and model capacity can significantly impact the model's ability to learn different types of capabilities (Hoffmann et al., 2022). However, most of this work has focused on large models, leaving open questions about how these relationships manifest in smaller architectures.

2.5 Efficient and Small Language Models

Research on efficient language models has seen renewed interest with the success of smaller but capable architectures. Notable examples include Microsoft's Phi models (Gunasekar et al., 2023), which achieved strong reasoning capabilities at 1.3B parameters (50.6% on HumanEval), and TinyLlama (Zhang et al., 2024), which adapted the Llama architecture to 1.1B parameters. The SmolLM family (Allal et al., 2024) represents a significant advance in this direction, demonstrating that carefully trained compact models can achieve competitive performance on a wide range of tasks while remaining deployable on consumer hardware.

3 Supervised Finetuning

3.1 Dataset

To ensure fair evaluation, we analyzed the contamination levels of various benchmarks in the SFT dataset (allenai/tulu-3-sft-mixture). As shown in Table 1, most benchmarks exhibit minimal contamination rates below 1.5%, with many showing zero contamination. The highest contamination rate was observed in PopQA at 7.21%, while critical evaluation benchmarks like GSM8K, IFEval, and AGI Eval showed negligible to zero contamination, ensuring reliable performance measurements.

Benchmark	**Contamination**
cais/mmlu	1.34%
openai/openai_humaneval	0.00%
openai/gsm8k	0.08%
ucinlp/drop	0.20%
lighteval/MATH	0.06%
google/IFEval	0.00%
akariasai/PopQA	7.21%
tatsu-lab/alpaca_eval	1.37%
lukaemon/bbh	0.02%
truthfulqa/truthful_qa	1.47%
allenai/wildguardmix	0.06%
allenai/wildjailbreak	0.00%
TIGER-Lab/MMLU-Pro	0.93%
Idavidrein/gpqa	0.00%
lighteval/agi_eval_en	0.00%
bigcode/bigcodebench	0.00%
deepmind/math_dataset	0.00%

Table 1: Contamination of benchmarks in the SFT dataset used *allenai/tulu-3-sft-mixture*

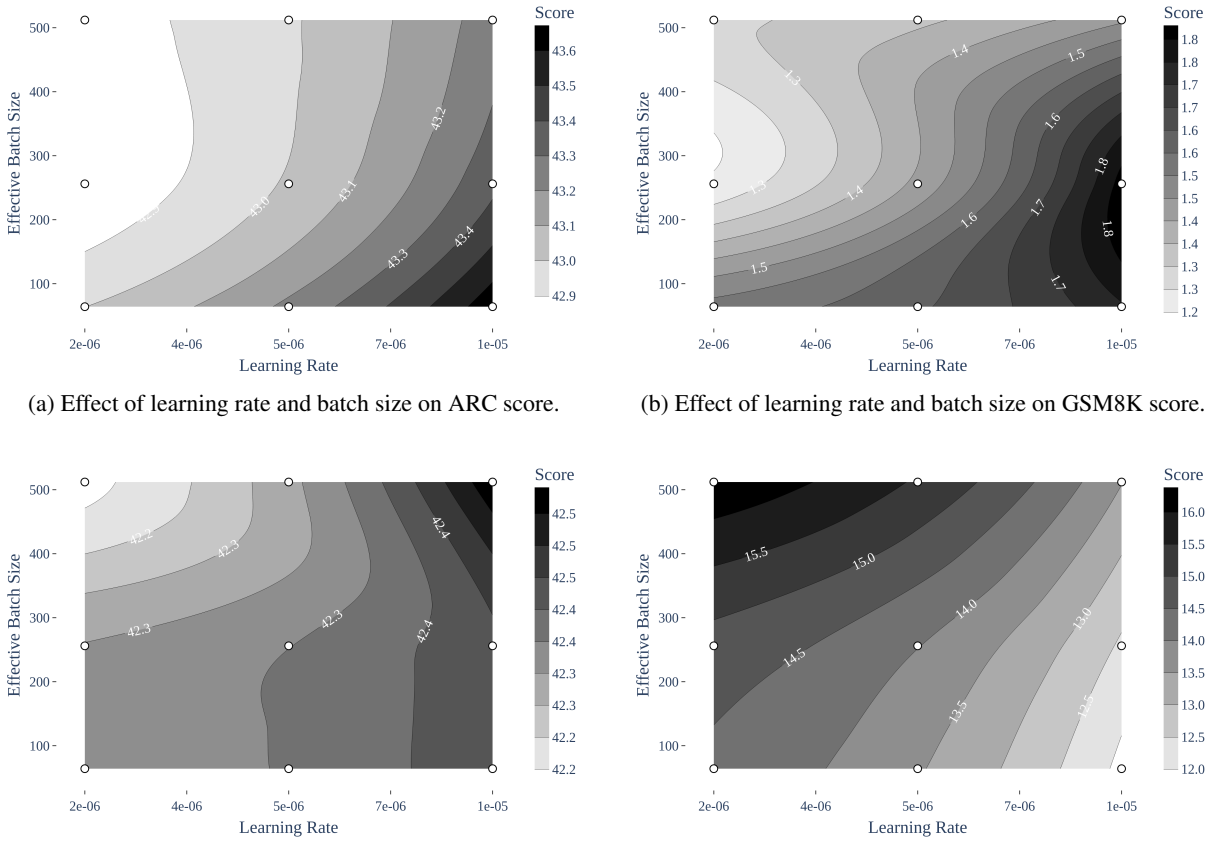
3.2 Training

Foundation models have demonstrated remarkable capabilities across various tasks, yet their performance can be significantly influenced by the choice of hyperparameters during SFT. While work by (Smith et al., 2018) established foundational principles for batch size scaling in neural networks, and (McCandlish et al., 2018) provided theoretical frameworks for understanding large-batch training, recent work by (Masters and Luschi, 2018) suggests that the conventional wisdom about batch size scaling may need reconsideration, particularly for smaller models.

Our experiments with SmolTulu variants build upon hyperparameter configurations established by AllenAI's extensive ablation studies for Tulu 3. As shown in Table 2, there is a notable pattern in the learning rate to batch size ratio (LR/BS) across model scales: as models grow larger, this ratio tends to decrease. This aligns with the scaling laws observed by (Kaplan et al., 2020) and the compute-optimal training strategies described in (Hoffmann et al., 2022). The Tulu 3 70B model uses the smallest ratio of 0.016×10^{-6}, while our SmolTulu SFT-1130 variant employs a significantly larger ratio of 11.25×10^{-6}.

Hyperparameter	SmolTulu SFT-1130	SmolTulu SFT-1207	Tulu 3 SFT 8b	Tulu 3 SFT 70b
Learning Rate (LR)	9.0×10^{-5}	3.1×10^{-6}	5.0×10^{-6}	2.0×10^{-6}
Batch Size (BS)	8	32	128	128
$\frac{LR}{BS} \times 10^6$	11.25	0.097	0.039	0.016

Table 2: SFT hyperparameter selection



(a) Effect of learning rate and batch size on ARC score.

(b) Effect of learning rate and batch size on GSM8K score.

(c) Effect of learning rate and batch size on HellaSwag score.

(d) Effect of learning rate and batch size on IFEval score.

Figure 1: Contour analysis of learning rate and batch size effects on different evaluation metrics during supervised finetuning of SmolLM2-135M. The color scales represent scores for each metric, with black indicating higher performance. The patterns reveal task-dependent optimal ratios between learning rate and batch size.

Metric	SmolTulu SFT-1130	SmolTulu SFT-1207	SmolLM2 1.7B-Instruct
ARC (Average)	51.0	**55.6**	51.7
BBH (3-shot)	**34.7**	34.0	32.2
GSM8K (5-shot)	**49.0**	42.8	48.2
HellaSwag	61.5	**67.5**	66.1
IFEval (Average)	**61.0**	47.8	56.7
MMLU-Pro (MCF)	17.6	17.9	**19.3**
PIQA	72.7	**76.9**	74.4

Table 3: Performance comparison of SFT models

3.3 Results & Discussion

The supervised finetuning experiments reveal complex relationships between optimization dynamics and model capabilities that evolve with model scale. Our initial ablation study using a 135M parameter model demonstrates clear task-dependent patterns in the relationship between learning rate and batch size ratios.

As shown in Figure 1a and Figure 1b, reasoning tasks like ARC and GSM8K consistently benefit from higher learning rate to batch size ratios. This aligns with theoretical work by (Keskar et al., 2017). The improvement is particularly pronounced for GSM8K, where performance increases monotonically with the learning rate to batch size ratio.

Conversely, at the 135M scale, pattern recognition tasks show markedly different behavior. Figure 1c and Figure 1d reveal that HellaSwag and IFEval achieve optimal performance with lower

learning rate to batch size ratios. This contrast suggests that at smaller scales, different types of learning may benefit from fundamentally different optimization dynamics, supporting (Shallue et al., 2019)'s observation that optimal batch sizes vary significantly between workloads.

However, these relationships become more nuanced at larger model scales. In our 1.7B parameter model, while GSM8K continues to benefit from higher ratios (achieving 51.6% with SmolTulu-1130's high ratio versus 44.7% with SmolTulu-1207's lower ratio), the pattern for other tasks shifts. Notably, IFEval performance improves with higher ratios (67.7% vs 56.6%), while ARC and MMLU-Pro show optimal performance with lower ratios (57.1% and 19.1% respectively).

This scale-dependent shift in optimization dynamics aligns with (Masters and Luschi, 2018)'s argument that the relationship between batch size and learning rate fundamentally changes with model capacity. We hypothesize that at 135M parameters, the limited model capacity forces a strict trade-off between different types of learning, requiring distinct optimization strategies for reasoning versus pattern recognition. At 1.7B parameters, the increased capacity appears to enable more flexible learning dynamics, allowing both GSM8K and IFEval to benefit from aggressive optimization while tasks like ARC and MMLU-Pro benefit from more conservative approaches.

These findings suggest that the conventional wisdom about learning rate and batch size relationships derived from large-scale training may need significant adaptation for smaller models. The interaction between model capacity and optimal optimization strategy appears more complex previously thought.

4 Direct Preference Optimization

4.1 Dataset

We conducted contamination analysis on the DPO dataset (allenai/llama-3.1-tulu-3-8b-preference-mixture) to verify evaluation fairness. Table 4 shows consistently low contamination rates across benchmarks, with most falling below 1%. The highest contamination was found in PopQA at 2.72%, while crucial benchmarks like GSM8K, IFEval, and BBH maintained zero contamination, enabling trustworthy assessment of model improvements through preference optimization.

Benchmark	Contamination
cais/mmlu	0.69%
openai/openai_humaneval	0.00%
openai/gsm8k	0.00%
ucinlp/drop	0.07%
lighteval/MATH	0.02%
google/IFEval	0.00%
akariasai/PopQA	2.72%
tatsu-lab/alpaca_eval	1.24%
lukaemon/bbh	0.00%
truthfulqa/truthful_qa	0.61%
allenai/wildguardmix	0.06%
allenai/wildjailbreak	0.00%
TIGER-Lab/MMLU-Pro	0.36%
Idavidrein/gpqa	0.00%
lighteval/agi_eval_en	0.00%
bigcode/bigcodebench	0.00%
deepmind/math_dataset	0.00%

Table 4: Contamination of benchmarks in the DPO dataset used *allenai/llama-3.1-tulu-3-8b-preference-mixture*

4.2 Training

We employed Direct Preference Optimization (DPO) for preference learning while adapting the hyperparameters for our smaller model scale, using the same pipeline as described in Tulu 3 (Lambert et al., 2024a). DPO allows direct optimization of the policy without requiring a separate reward model (Rafailov et al., 2024). The core objective function is shown in equation 1.

We used the length-normalized variant of DPO, which showed superior performance in Tulu 3's ablations, where the log probabilities are normalized by sequence length.

- Pre-computing and caching log probabilities from the reference model instead of keeping it in memory (Hu et al., 2024)
- Performing separate forward passes for chosen and rejected sequences rather than concatenating them

However, our experiments revealed that smaller models benefit from different hyperparameter configurations than their larger counterparts. As shown in Table 5, we maintained higher learning rate to batch size ratios compared to Tulu 3's models, with SmolTulu DPO-1130 using a ratio approximately 17 times larger than Tulu 3 8B's.

$$\max_{\pi_\theta} \mathbb{E}_{y_c, y_r \sim \mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_c|x)}{\pi_{\text{ref}}(y_c|x)} - \beta \log \frac{\pi_\theta(y_r|x)}{\pi_{\text{ref}}(y_r|x)} \right) \right] \tag{1}$$

Figure 2: Direct Preference Optimization objective function, where π_{ref} is the reference model (our SFT model) and β controls KL divergence from the reference model.

Hyperparameter	**SmolTulu DPO-1130**	**SmolTulu DPO-1207**	**Tulu 3 DPO 8b**	**Tulu 3 DPO 70b**
Learning Rate (LR)	8.0×10^{-7}	5×10^{-7}	5.0×10^{-7}	2.0×10^{-7}
Batch Size (BS)	12	32	128	128
$\frac{LR}{BS} \times 10^7$	0.667	0.156	0.039	0.016

Table 5: DPO hyperparameter selection

For DPO-1130, we used a learning rate of 8.0×10^{-7} and batch size of 12, while DPO-1207 used a more conservative learning rate of 5.0×10^{-7} with batch size 32. Both variants used:

- Maximum sequence length of 2,048 tokens
- KL penalty coefficient $\beta = 5$
- Linear learning rate schedule with 0.1 warmup ratio
- Single training epoch

Our experimentation with higher learning rate to batch size ratios was motivated by the hypothesis that smaller models may require larger per-example updates to effectively learn from preference data, particularly for complex reasoning tasks where they lack the inherent capacity of larger models.

4.3 Results & Discussion

Metric	**SmolTulu DPO-1130**	**SmolTulu DPO-1207**	**SmolLM2 1.7B-Instruct**
ARC (Average)	51.5	**57.1**	51.7
BBH (3-shot)	**33.8**	**34.2**	32.2
GSM8K (5-shot)	**51.6**	44.7	48.2
HellaSwag	61.1	64.2	**66.1**
IFEval (Average)	**67.7**	56.6	56.7
MMLU-Pro (MCF)	17.4	19.1	**19.3**
PIQA	72.2	**76.4**	74.4

Table 6: Performance comparison of DPO models

The DPO experiments yielded notable improvements over the SFT baseline across several key metrics. As shown in Table 6, SmolTulu DPO-1130, using the higher learning rate to batch size ratio, achieved substantial gains on instruction following and mathematical reasoning tasks, reaching 67.7% on IFEval and 51.6% on GSM8K. Conversely, SmolTulu DPO-1207, employing a lower learning rate to batch size ratio more similar to larger models, showed stronger performance on pattern recognition tasks like ARC (57.1%) and PIQA (76.4%).

However, our exploration of hyperparameter configurations was significantly constrained by computational resources. Given that DPO builds directly upon the SFT model, the initial policy's convergence properties may significantly influence subsequent preference learning. A comprehensive understanding would require extensive ablation studies across different SFT checkpoints.

5 Reward Modelling

5.1 Dataset

Our reward models (RM) were trained on the same preference dataset used in the DPO stage, combining UltraFeedback with additional synthetic preference data generated through the Tulu 3 pipeline. We maintained rigorous contamination controls across all evaluation benchmarks, as detailed in earlier sections.

5.2 Training

Following the Tulu 3 methodology (Lambert et al., 2024a), we trained our reward models using the standard pairwise preference learning objective. Given a preference dataset $\mathcal{D}$ consisting of prompts x and two responses per prompt (y, y'), where one response is chosen y_c and one is rejected y_r, the reward model r_ϕ is trained to maximize:

$$\max_{r_\phi} \mathbb{E}_{(x, y_c, y_r) \sim \mathcal{D}} [\log \sigma(r_\phi(x, y_c) - r_\phi(x, y_r))] \tag{2}$$

where σ is the logistic function. This objective maximizes the difference between rewards, with this difference representing the log-likelihood that

Hyperparameter	SmolTulu RM-1130	SmolTulu RM-1207	Tulu 3 DPO 8b
Learning Rate (LR)	4.0×10^{-5}	7.5×10^{-7}	5.0×10^{-7}
Batch Size (BS)	4	8	128
$\frac{LR}{BS} \times 10^7$	100	0.938	0.039

Table 7: Reward model hyperparameter selection

y_c will be preferred over y_r. We used the same several key implementation details from Tulu 3, except for ones that involved batch size and learning rate, where our changes can be seen in Table 7.

5.3 Results & Discussion

Metric	SmolTulu RM-1130	SmolTulu RM-1207	Tulu 3 8b RM
RB Chat	*94.13*	83.52	**96.27**
RB Chat Hard	43.64	*44.74*	**55.92**
RB Safety	*75.54*	64.59	**84.05**
RB Reasoning	*68.01*	54.71	**76.50**
RB Average	*72.43*	58.59	**81.34**
UFB	*73.17*	61.66	**77.34**

Table 8: Performance comparison of reward models, where UFB is the *test_prefs* split of *allenai/ultrafeedback_binarized_cleaned* and RB is RewardBench.

The reward modeling experiments revealed interesting patterns consistent with our findings from SFT and DPO stages. SmolTulu RM-1130, employing a much larger learning rate to batch size ratio, demonstrated strong performance across various metrics on RewardBench (RB) (Lambert et al., 2024b), achieving 94.13% on standard chat evaluation and 75.54% on safety assessments, as seen in Table 8. This pattern of strong relative performance extends across other metrics, with SmolTulu RM-1130 achieving 73.17% accuracy on UltraFeedback benchmark test preferences, falling only 4.17 percentage points short of Tulu 3's 77.34% despite using approximately 21% of the parameters. Following (Shallue et al., 2019)'s framework, these results suggest that reward modeling may scale more gracefully to smaller architectures than previously assumed, particularly when using appropriately adapted optimization strategies.

The substantial performance gap between RM-1130 and RM-1207 (72.43% vs 58.59% on RB reinforces our earlier findings about the importance of learning rate to batch size ratios in smaller models. The higher ratio used in RM-1130 appears to be particularly crucial for reward modeling, where the task of learning preference relationships may benefit from larger per-example updates and more frequent gradient computations. However, establishing the precise nature of this relationship would require more extensive ablation studies, which we leave to future work with greater computational resources.

6 Reinforcement Learning with Verifiable Rewards

While our initial experiments with RLVR showed promise, the computational requirements for thorough hyperparameter exploration proved prohibitive.

Our preliminary investigations suggest that the relationship between learning rate and batch size may be particularly complex in the RLVR setting, where the sparse binary reward signal introduces additional optimization challenges for smaller models. However, establishing concrete findings would require extensive ablation studies beyond our current computational resources. We leave comprehensive exploration of these dynamics to future work with greater computational capacity.

7 Limitations

While our work demonstrates promising results in adapting the Tulu 3 pipeline to smaller models, several important limitations should be noted:

Base Model Dependencies Our findings are specific to SmolLM2 and may not generalize to other small models, particularly those with different pretraining approaches or architectural choices.

Multi-Stage Optimization Understanding The relationship between optimization choices in different training stages (SFT, DPO, RM, RLVR) remains poorly understood, especially in the context of smaller models. While we observed consistent benefits from higher learning rate to batch size ratios, the theoretical foundations for this interaction across training stages requires further investigation.

Metric	SmolTulu DPO-1130	SmolTulu DPO-1207	SmolTulu SFT-1130	SmolTulu SFT-1207	SmolLM2 1.7B-Instruct	Llama-3.2 1B-Instruct	Qwen2.5 1.5B-Instruct
ARC (Average)	51.5	**57.1**	51.0	55.6	51.7	41.6	46.2
BBH (3-shot)	33.8	34.2	34.7	34.0	32.2	27.6	**35.3**
GSM8K (5-shot)	**51.6**	44.7	49.0	42.8	48.2	26.8	42.8
HellaSwag	61.1	64.2	61.5	**67.5**	66.1	56.1	60.9
IFEval (Average)	**67.7**	56.6	61.0	47.8	56.7	53.5	47.4
MMLU-Pro (MCF)	17.4	19.1	17.6	17.9	19.3	12.7	**24.2**
PIQA	72.2	76.4	72.7	**76.9**	74.4	72.3	73.2

Table 9: A comparison against a wider selection of models

8 Conclusion

We have demonstrated that careful adaptation of modern post-training techniques can yield strong results even at significantly smaller model scales. We found that smaller models may require substantially different optimization dynamics than their larger counterparts to achieve optimal performance. Our empirical results align with theoretical frameworks from optimization literature, suggesting that higher ratios can help compensate for limited model capacity, particularly on complex reasoning tasks.

The resulting model, SmolTulu, achieves state-of-the-art performance among sub-2B parameter models on instruction following while maintaining strong mathematical reasoning capabilities as seen in Table 9. These results suggest that the effectiveness of post-training pipelines like Tulu 3 can extend to much smaller scales when properly adapted. Our findings indicate that optimal training strategies may need to vary significantly based on both model scale and target capabilities.

Looking forward, we believe this work opens up promising directions for making high-quality language models more accessible and deployable in resource-constrained environments. Future work investigating adaptive optimization strategies that account for both model scale and task requirements could further advance this goal. Additionally, developing theoretical frameworks that specifically address the interaction between model capacity and optimization dynamics could help establish more principled approaches to training smaller models.

9 Acknowledgements

Thank you Faisal Alhejary, Abdulmajeed Alrowaithy, Tariq Aljaber, Abdulaziz Albuainain, and Salman Alsubaihi for providing me with their support. Thank you AllenAI and Huggingface for your continual contributions to open-source.

References

Loubna Ben Allal, Anton Lozhkov, Elie Bakouch, Gabriel Martín Blázquez, Lewis Tunstall, Agustín Piqueres, Andres Marafioti, Cyril Zakka, Leandro von Werra, and Thomas Wolf. 2024. Smollm2 - with great data, comes great performance.

Wei-Lin Chiang, Zhuohan Li, Zi Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E. Gonzalez, Ion Stoica, and Eric P. Xing. 2023. Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality.

Priya Goyal, Piotr Dollár, Ross Girshick, Pieter Noordhuis, Lukasz Wesolowski, Aapo Kyrola, Andrew Tulloch, Yangqing Jia, and Kaiming He. 2018. Accurate, large minibatch sgd: Training imagenet in 1 hour.

Suriya Gunasekar, Yi Zhang, Jyoti Aneja, Caio César Teodoro Mendes, Allie Del Giorno, Sivakanth Gopi, Mojan Javaheripi, Piero Kauffmann, Gustavo de Rosa, Olli Saarikivi, Adil Salim, Shital Shah, Harkirat Singh Behl, Xin Wang, Sébastien Bubeck, Ronen Eldan, Adam Tauman Kalai, Yin Tat Lee, and Yuanzhi Li. 2023. Textbooks are all you need.

Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. 2022. Training compute-optimal large language models.

Jian Hu, Xibin Wu, Zilin Zhu, Xianyu, Weixun Wang, Dehao Zhang, and Yu Cao. 2024. Openrlhf: An easy-to-use, scalable and high-performance rlhf framework.

Hamish Ivison, Yizhong Wang, Valentina Pyatkin, Nathan Lambert, Matthew Peters, Pradeep Dasigi, Joel Jang, David Wadden, Noah A. Smith, Iz Beltagy, and Hannaneh Hajishirzi. 2023. Camels in a changing climate: Enhancing lm adaptation with tulu 2.

Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray,

Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling laws for neural language models.

Amirhossein Kazemnejad, Milad Aghajohari, Eva Portelance, Alessandro Sordoni, Siva Reddy, Aaron Courville, and Nicolas Le Roux. 2024. Vineppo: Unlocking rl potential for llm reasoning through refined credit assignment.

Nitish Shirish Keskar, Dheevatsa Mudigere, Jorge Nocedal, Mikhail Smelyanskiy, and Ping Tak Peter Tang. 2017. On large-batch training for deep learning: Generalization gap and sharp minima.

Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, Hamish Ivison, Faeze Brahman, Lester James V. Miranda, Alisa Liu, Nouha Dziri, Shane Lyu, Yuling Gu, Saumya Malik, Victoria Graf, Jena D. Hwang, Jiangjiang Yang, Ronan Le Bras, Oyvind Tafjord, Chris Wilhelm, Luca Soldaini, Noah A. Smith, Yizhong Wang, Pradeep Dasigi, and Hannaneh Hajishirzi. 2024a. Tülu 3: Pushing frontiers in open language model post-training.

Nathan Lambert, Valentina Pyatkin, Jacob Morrison, LJ Miranda, Bill Yuchen Lin, Khyathi Chandu, Nouha Dziri, Sachin Kumar, Tom Zick, Yejin Choi, Noah A. Smith, and Hannaneh Hajishirzi. 2024b. Rewardbench: Evaluating reward models for language modeling.

Dominic Masters and Carlo Luschi. 2018. Revisiting small batch training for deep neural networks.

Sam McCandlish, Jared Kaplan, Dario Amodei, and OpenAI Dota Team. 2018. An empirical model of large-batch training.

Yu Meng, Mengzhou Xia, and Danqi Chen. 2024. Simpo: Simple preference optimization with a reference-free reward. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. 2022. Training language models to follow instructions with human feedback.

Du Phan, Matthew D. Hoffman, David Dohan, Sholto Douglas, Tuan Anh Le, Aaron Parisi, Pavel Sountsov, Charles Sutton, Sharad Vikram, and Rif A. Saurous. 2023. Training chain-of-thought via latent-variable inference.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. 2024. Direct preference optimization: Your language model is secretly a reward model.

Christopher J. Shallue, Jaehoon Lee, Joseph Antognini, Jascha Sohl-Dickstein, Roy Frostig, and George E. Dahl. 2019. Measuring the effects of data parallelism on neural network training.

Samuel L. Smith, Pieter-Jan Kindermans, Chris Ying, and Quoc V. Le. 2018. Don't decay the learning rate, increase the batch size.

Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. 2023. Stanford alpaca: An instruction-following llama model. https://github.com/tatsu-lab/stanford_alpaca.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. 2023. Llama 2: Open foundation and fine-tuned chat models.

Lewis Tunstall, Edward Beeching, Nathan Lambert, Nazneen Rajani, Kashif Rasul, Younes Belkada, Shengyi Huang, Leandro von Werra, Clémentine Fourrier, Nathan Habib, Nathan Sarrazin, Omar Sanseviero, Alexander M. Rush, and Thomas Wolf. 2023. Zephyr: Direct distillation of lm alignment.

Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. 2022. Star: Bootstrapping reasoning with reasoning.

Peiyuan Zhang, Guangtao Zeng, Tianduo Wang, and Wei Lu. 2024. Tinyllama: An open-source small language model.

Yao Zhao, Rishabh Joshi, Tianqi Liu, Misha Khalman, Mohammad Saleh, and Peter J. Liu. 2023. Slic-hf: Sequence likelihood calibration with human feedback.

A Language Distribution in Datasets

Using *XLM-RoBERTa* for language detection, we can get an estimate for the presence of each language in a given dataset.

B RLVR Dataset Contamination

We included the contamination results of our intended RLVR dataset for reference.

Language	Presence (%)
English	83.13
Hindi	3.79
Swahili	2.02
Russian	2.00
Spanish	1.15
Arabic	0.98
Chinese	0.94
Turkish	0.87
Urdu	0.78
Portuguese	0.77
Vietnamese	0.64
Japanese	0.63
French	0.66
Bulgarian	0.33
Italian	0.32
Dutch	0.31
Polish	0.25
German	0.23
Thai	0.10
Greek	0.09

Table 10: Language distribution in SFT dataset.

Language	Presence (%)
English	86.24
Hindi	2.23
Russian	2.03
French	1.42
Spanish	1.40
Chinese	1.37
Urdu	0.68
Swahili	0.65
German	0.58
Japanese	0.57
Portuguese	0.54
Arabic	0.51
Turkish	0.42
Vietnamese	0.33
Italian	0.32
Polish	0.22
Dutch	0.18
Bulgarian	0.18
Thai	0.10
Greek	0.04

Table 11: Language distribution in DPO / RM dataset.

Language	Presence (%)
English	94.80
French	1.29
Spanish	1.04
Chinese	0.66
German	0.55
Russian	0.48
Japanese	0.40
Hindi	0.23
Polish	0.10
Portuguese	0.10
Dutch	0.08
Urdu	0.07
Bulgarian	0.07
Italian	0.05
Turkish	0.03
Arabic	0.03
Vietnamese	0.02
Swahili	0.00

Table 12: Language distribution in RLVR dataset.

Benchmark	Contamination
cais/mmlu	0.65%
openai/openai_humaneval	0.00%
openai/gsm8k	0.00%
ucinlp/drop	0.00%
lighteval/MATH	0.24%
google/IFEval	0.00%
akariasai/PopQA	0.45%
tatsu-lab/alpaca_eval	0.12%
lukaemon/bbh	0.00%
truthfulqa/truthful_qa	0.12%
allenai/wildguardmix	0.00%
allenai/wildjailbreak	0.00%
TIGER-Lab/MMLU-Pro	0.66%
Idavidrein/gpqa	0.00%
lighteval/agi_eval_en	0.00%
bigcode/bigcodebench	0.00%
deepmind/math_dataset	0.00%

Table 13: Contamination of benchmarks in the RLVR dataset *allenai/RLVR-GSM-MATH-IF-Mixed-Constraints*